

File Operations: Open, Read, Write, Close

1. **Open:** Use the built-in `open()` function. It returns a file object.
 - `file_object = open('filename', 'mode')`
2. **Read/Write:** Use methods on the file object (e.g., `read()`, `write()`).
3. **Close:** Use the `.close()` method to release the file handle, freeing system resources. Failing to close files can lead to data corruption or resource exhaustion.

File Modes

Mode	Description	Real-time Use Case
'r'	Read (Default). Error if file doesn't exist.	Reading a configuration file.
'w'	Write . Creates the file if it doesn't exist, overwrites content if it does.	Generating a report from scratch.
'a'	Append . Creates the file if it doesn't exist, adds new data to the end.	Writing new log entries.
'r+'	Read and Write. Pointer at the beginning.	Modifying the start of a file.
'w+'	Write and Read. Overwrites or creates.	Creating a temp file, writing, and then immediately reading back.
't'	Text mode (Default). Handles encoding/decoding.	Reading a standard text file.
'b'	Binary mode. Used for images, executables, etc.	Reading/writing image data or encrypted files.

Reading Methods

Method	Description	Real-time Use Case
<code>read(size)</code>	Reads the entire file content as a single string, or up to <code>size</code> bytes/characters.	Loading a small configuration file into a single variable.
<code>readline()</code>	Reads a single line from the file.	Processing a file line-by-line in a loop.

Method	Description	Real-time Use Case
readlines()	Reads all lines into a list of strings , where each element is one line.	Getting all user names from a delimited text file.

Python

Real-time Example: Reading a Log File

```
try:
    file = open("application.log", 'r')
    first_line = file.readline()
    all_content = file.read() # Reads remaining content from current position
    print(f"First Line: {first_line.strip()}")
    # print(f"Remaining Content: {all_content}")
```

```
except FileNotFoundError:
    print("Error: application.log not found.")
```

```
finally:
    if 'file' in locals() and not file.closed:
        file.close()
```

Writing Methods

Method	Description	Real-time Use Case
write(string)	Writes the specified string to the file. Returns the number of characters written.	Writing a single message or piece of data.
writelines(list_of_strings)	Writes a list of strings to the file. Does not automatically add newlines (\n).	Writing a list of processed results or a CSV file.

Python

Real-time Example: Writing processed data to a new CSV file

```
data_to_write = [
    "ID,Name,Status\n",
    "1,Alice,Active\n",
    "2,Bob,Inactive\n"
]
```

```
try:
    file = open("processed_data.csv", 'w')
    file.writelines(data_to_write)
    print("Data written to processed_data.csv")
```

```
finally:
    file.close()
```

The with statement and Context Managers (Recommended)